FORMAL METHODS REPORT

Lido
SRv3 Accounting Proof Closure

Executable evidence package for Lido's SRv3 economic accounting state machine. The selected P0 properties are checked over a focused Verity-style model with explicit source anchors and assumptions.

Version	Private Report v1.0
Date	June 2026
Commit	`PR-1811 / d088bbc2`
Reviewers	LFG Labs

Executable evidence for smart-contract state-machine review.

lfglabs.dev

Confidentiality and result status

This document is a **private executable evidence package** for the selected SRv3 accounting proof-closure lane. It connects the report claims to local model files, reference tests, proof target files, and reproducibility commands for PR #1811 at the commit named on the cover.

The report is intentionally narrow. It covers the economic state machine modeled for deposits, reserves, module balances, accepted reports, rewards, fees, and status gates. It does not claim full deployed-system equivalence, oracle truthfulness, cryptographic correctness, governance correctness, or exact proxy/storage refinement beyond the named assumptions.

Every selected target row is backed by a source-code cross-reference, trust-boundary entry, local Verity-style model check, and rebuild command. These artifacts are executable bounded checks over the focused economic model, not unbounded Lean theorems and not full deployed-system refinement proofs.

> **Distribution**
> Private review channel for Lido protocol contributors and selected technical reviewers. Public release, forum excerpts, or website publication should keep the same bounded-check language unless stronger theorem-checker artifacts are added and independently rebuilt.

Contents

1 Executive summary

Question answered by this report

If PR #1811 changes how Lido accounts for deposits, reserves, module balances, and rewards, can the selected accounting rules be stated precisely enough that a machine can check them in a reproducible focused model? This report package gives an executable local answer for the six selected P0 (highest-priority) properties over the focused economic model, under the named assumptions in Section 3.

In this repository, "executable" has a specific meaning: the model target is listed, all premises not discharged inside the model are registered by assumption ID, and `make test` plus `make prove` recheck the target set from a clean checkout.

The story in one page

The core safety question is simple enough to ask without Solidity or Lean: *when ETH and validator balances move through SRv3 accounting, can any value be spent from the wrong place, counted twice, rewarded from stale data, or sent through a disabled path?* The report answers that question by turning each informal concern into a small conservation law.

A useful mental model is three ledgers that must agree:

1. The **buffer ledger** says which ETH is depositable and which ETH is reserved for withdrawals.
2. The **router ledger** says how much ETH is pulled for deposits and how validator balances are summed across staking modules.
3. The **reward ledger** says which accepted balances may be used to compute rewards, fees, and recipients.

An "accepted report" means oracle-provided module-balance data that has passed the modeled SRv3 validation checks and is therefore allowed to update accounting state. The proof lane checks that PR #1811 preserves the three ledgers through those modeled transitions. It is not trying to prove every operational fact about deployment, cryptography, governance, gas, or the consensus layer.

Why this is more than testing

A test asks, "does this example work?" An executable model check asks, "does this rule survive the modeled transition and fixture or bounded enumeration that the artifact names?" That is why the report spends so much effort naming the model, bounds, and assumptions: they define exactly what the local command checks.

The executable claim this report package supports is:

> For the focused SRv3 economic state machine, the local model tests and bounded proof harness preserve deposit conservation, reserve separation, module-balance conservation, report-before-reward consistency, reward/fee bounds, and flow-specific status gating over the named fixtures and enumerated domains.

This is not a claim that every deployed-system behavior is proved. The exact boundary is part of the result: cryptographic witnesses, consensus truthfulness, governance configuration, and deployment refinement are named in Section 3.

How to read the proof

Read each row below as a three-step trace. First, the Solidity-facing model names the transition being studied. Second, the property states what must not go wrong in the focused model. Third, the local harness checks the Verity-style model against the explicit assumptions.

The matrix answers six plain questions: what ETH is spendable, what deposits consume, what balances add up to, what report rewards use, how fees are bounded, and which module states block economic flow.

Proof matrix

All six rows below have executable artifacts in `proofs/`, `verity/`, and `tests/verity/`.

ID	PR #1811 surface	Executable property
SRV3-P1	Lido buffer allocation	Withdrawal-reserved ETH is excluded from the depositable resource, so deposit spending cannot borrow from withdrawal liquidity.
SRV3-P2	Router deposit path	Router-pulled ETH equals 32 ETH times actual deposits and is fully sunk into CL deposits, with no modeled residual held by the router.
SRV3-P3	Module balances	Router-wide validator balance is exactly the sum of accepted per-module balances, so the aggregate cannot drift away from the module ledger.
SRV3-P4	Oracle report ordering	Module fee-distribution weights come from the current accepted module-balance state, not from stale or partially applied reports.
SRV3-P5	Rewards and fees	Module fee-distribution weights and recipient alignment use accepted module balances; total reward and fee magnitude remain accepted report or vault-context inputs under assumptions.
SRV3-P6	Status gates	Non-active or deposit-paused modules receive no modeled deposit allocation; stopped modules receive no module-side reward or fee allocation.

Review posture

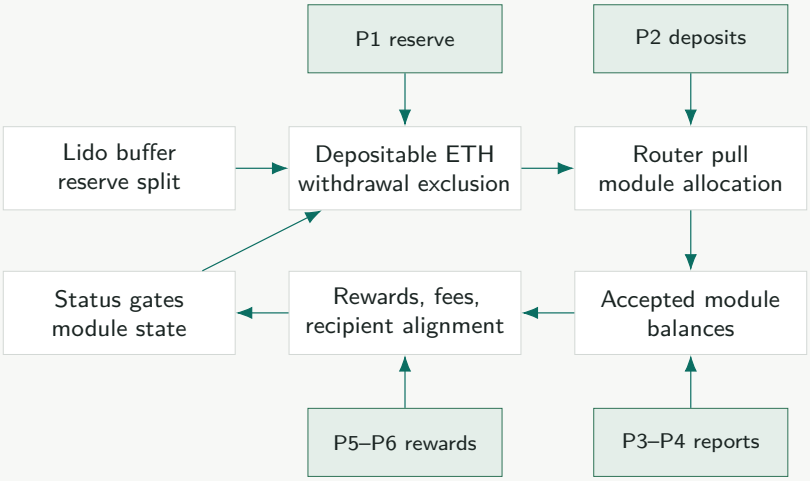
This report package is written for two readers at once. A protocol reviewer can follow the ledgers and the trust boundary without reading code. A formal-methods reviewer can check whether the target IDs, assumptions, and rebuild gates support the advertised local executable claim.

2 What the proof closes

A local proof target is closed in this package when the executable model check passes, the model and source anchors are listed, and the remaining premises are named as assumptions. This section shows the six accounting claims in three forms: a system map, a plain-language story, and artifact rows for the evidence package.

Modeled architecture

The proof model follows the accounting path where PR #1811 concentrates fund-safety risk. In plain terms, the model follows ETH as it moves from the Lido buffer, through the router, into staking-module deposits, and then into later balance reports and reward calculations. Each proof asks whether that flow preserves the accounting rule it is supposed to preserve. External contracts are summarized only at the interfaces used by those transitions.



From intuition to target

Read the six properties as one accounting story. P1 protects ETH reserved for withdrawals. P2 checks that initial deposits pull exactly the ETH they should. P3 checks that the router's total balance equals the module balances it accepted. P4 makes module fee weights depend on the accepted report, not an old or partial one. P5 bounds reward and fee movement. P6 blocks non-active or paused modules from deposit allocation and stopped modules from module-side reward or fee allocation.

Each invariant below has two layers. The first sentence says why the property matters in ordinary accounting language. The formula then states the same idea in a form that can be checked over all modeled inputs. You can read the formulas like receipts: they name the money bucket before and after a move, then check that the totals still match. The symbols below only stand for modules, balances, reserves, and reports already described in the ledger story.

Let M be the finite set of staking modules in router order. For a module $m \in M$, let bal_m be its accepted validator balance in gwei, status_m its module status, and $\mathsf{moduleRecipient}_m$ its module reward recipient in the modeled reward step. Let B_R be the router validator balance, T the buffered ETH, R_D the stored deposit reserve, and R_W the withdrawal-reserved buffer. The executable checks cover the named fixtures and the bounded enumeration in `verity/src/verity_srv3/prove.py`. Each accepted report must still cover

the current router module set in router order, with unique module IDs, matching array lengths, and the freshness and range premises named in the target file.

P1: reserve separation.

$$d = \min(T, R_D) \qquad w = \min(T - d, R_W) \qquad u = T - d - w$$

$$\mathsf{depositable}(T) = d + u$$

$$\forall a. \quad \mathsf{spendDeposit}(a) \Rightarrow a \le \mathsf{depositable}(T)$$

$$\mathsf{spendDeposit}(a) \Rightarrow \mathsf{usesWithdrawalReserve}(a, T, R_D, R_W) = 0$$

Plain reading: some ETH is saved for withdrawals, and deposits are not allowed to spend that saved amount. Deposit spend is bounded by the pre-spend deposit reserve plus unreserved allocation and does not consume the pre-spend effective withdrawal-reserved bucket. The executable target deliberately avoids a raw post-state equality for R_W; any post-state reserve fact belongs to the model's explicit reserve-recomputation rule.

P2: exact router pull.

$$\mathsf{pulledWei} = 32\,\mathsf{ETH} \cdot \mathsf{actualDeposits} \qquad \mathsf{routerEthAfter} = \mathsf{routerEthBefore}$$

$$\mathsf{actualDeposits} = \sum_{m \in M} \mathsf{allocatedDeposits}_m$$

Plain reading: across the modeled atomic initial-deposit transition, the router pulls exactly one 32 ETH unit per validator and immediately forwards it to the beacon deposit sink. Top-ups, if included in a later lane, are a separate gwei-aligned allocation target and are not instances of this 32 ETH exact-pull statement.

P3: module-balance conservation.

$$\mathsf{acceptedReport}(M, \vec{b}) \Rightarrow B_R' = \sum_{m \in M} \mathsf{bal}_m' = \sum_{i=0}^{|M|-1} \vec{b}_i$$

Plain reading: the router total must equal the sum of the module rows after an accepted report. The proof is over arbitrary finite module arrays, with module-ID uniqueness and report alignment named as assumptions rather than hidden bounds.

P4: report-before-reward consistency.

$$\mathsf{rewardStep}(s, s') \Rightarrow \exists r.\, \mathsf{acceptedAt}(r, s) \wedge \mathsf{rewardBase}(s) = \mathsf{balances}(r)$$

Plain reading: accepted module balances determine the module fee-distribution weights used by the modeled reward step. Total reward magnitude and protocol fee inputs are treated as accepted report or vault-context inputs subject to the listed range and freshness assumptions. In simpler terms, rewards should be split using the report the system just accepted, not a previous snapshot and not a half-updated table.

P5: bounded rewards and fee alignment.

$$0 \le \mathsf{moduleFee}_m \le \mathsf{totalReward} \cdot \mathsf{moduleFeeBps}_m / 10{,}000$$

$$\mathsf{moduleRecipient}_m' = \mathsf{moduleRecipient}_m \wedge \mathsf{rewardRecipient}(\mathsf{moduleFee}_m) = \mathsf{moduleRecipient}_m$$

Plain reading: fee amounts cannot exceed the configured basis-point bound, and each amount remains paired with the recipient for the module that earned it.

P6: status gating.

$$\text{status}_m \neq \text{active} \vee \text{depositsPaused}_m \Rightarrow \text{allocatedDeposits}_m = 0$$

$$\text{status}_m = \text{stopped} \Rightarrow \text{moduleReward}_m = 0$$

Plain reading: a stopped or paused module is not just marked differently in storage; that status changes the allowed flow. Deposit-paused or non-active modules receive zero modeled deposit allocation. Stopped modules receive zero module-side fee or reward allocation. Any residual protocol-fee treatment is stated separately from module recipient payment.

Report package traceability register

The next table is for auditability. The artifact path is the local file a reviewer can inspect or run. The assumption IDs say which facts the check is allowed to borrow from outside the model. The report does not ask the reader to trust prose alone; every claim has this chain.

ID	Executable artifact	Assumptions	Claim
SRV3-P1	`proofs/reserve-separation.md`	A-EXT-01; A-ARITH-05	Deposit spend cannot consume withdrawal-reserved ETH.
SRV3-P2	`proofs/deposit-conservation.md`	A-DEP-02; A-ID-04; A-ARITH-05	Pulled wei equals 32 ETH times actual deposits over all allocated modules.
SRV3-P3	`proofs/module-balance-conservation.md`	A-ORC-03; A-ID-04; A-ARITH-05	Accepted report application preserves router balance as the module sum.
SRV3-P4	`proofs/report-before-reward-consistency.md`	A-ORC-03; A-ID-04	Module fee-distribution weights read the latest accepted module-balance state.
SRV3-P5	`proofs/reward-correctness.md`	A-ORC-03; A-ARITH-05	Fee amounts are bounded and recipient/module arrays stay aligned.
SRV3-P6	`proofs/status-gating.md`	A-EXT-01; A-ID-04	Non-active or deposit-paused modules receive no modeled deposit allocation; stopped modules receive no module-side reward or fee allocation.

Evidence architecture

Layer	Artifact	Role
Solidity-facing summaries	`verity/src/verity_srv3/model.py`	Preserve the relevant array, loop, reserve, and call structure while summarizing external contracts through explicit interfaces.
Economic model	`verity/src/verity_srv3/model.py`	Defines modules, reserves, Wei/Gwei arithmetic, status flags, accepted reports, rewards, and fee state.
Proof obligations	`verity/src/verity_srv3/properties.py`	States each target over the focused transition model, with uniqueness, ordering, alignment, and freshness premises named in target files.
Trust-boundary report	`verity/targets/trust-boundary.json`	Machine-readable list of assumptions, target IDs, source links, retained Certora assumptions, and out-of-model claims.
Rebuild gate	`make test; make prove`	Rechecks all selected model tests and proof targets and emits `proofs/logs/proof-report.json`.

3 Trust boundary

A formal proof is not a proof of "everything the deployed system can ever do." It is a proof about a specific model. In plain terms, this model checks the accounting moves inside the selected SRv3 state machine; it does not prove that every outside system feeding that state machine is honest, available, or cryptographically sound. The boundary below is therefore part of the result, not a footnote.

What is inside the model

Inside	Why it is modeled
Reserve and buffer arithmetic	This is where depositable ETH must be separated from withdrawal-reserved ETH.
Router deposit accounting	This is where pulled ETH must match actual 32 ETH validator deposits.
Module-balance reports	This is where per-module balances become the router-wide accepted balance.
Rewards, fees, and recipients	This is where amount bounds and recipient alignment matter.
Module status gates	This is where non-active or paused modules stop receiving modeled deposit allocation, and stopped modules stop receiving module-side rewards or fees.

Named assumptions

An assumption is not a hidden conclusion. It is a fact this proof is allowed to start from. The point of naming assumptions is to show exactly which parts still need ordinary audit, deployment review, oracle review, or cryptographic review.

ID	Assumption	Meaning
A-EXT-01	External contract summaries	Lido buffer withdrawals, staking module callbacks, fee recipients, and beacon deposit sinks conform to the specified interfaces and mutate no unlisted SRv3 accounting state.
A-DEP-02	Beacon deposit sink	A modeled beacon deposit removes exactly the intended 32 ETH value from the router-side resource. Deposit-root, BLS, and SSZ correctness are not proved.
A-ORC-03	Accepted oracle reports	Once accepted by SRv3 validation, module-balance reports satisfy the listed shape, alignment, freshness, and range assumptions. Consensus-layer truthfulness is outside this report.

ID	Assumption	Meaning
A-ID-04	Module identity and ordering well-formedness	Module IDs are unique, report arrays align with those IDs where required, and sortedness obligations are enforced for ordered paths.
A-ARITH-05	Unit-conversion domain	Wei/Gwei and basis-point arithmetic is interpreted over the bounded unsigned integer domains used by the modeled transitions.

Explicit non-claims

Boundary	Treatment
BLS signatures, deposit roots, SSZ, Merkle proofs, EIP-4788	Named cryptographic and consensus-data boundaries, not part of the P0 accounting closure.
Exact packed-storage migration and proxy mechanics	Deferred to a deployment and migration lane; not used as proof evidence here.
Full governance role graph and parameter admissibility	Configuration is assumed to satisfy the stated range and role assumptions; unauthorized governance transitions are not modeled.
CL witness soundness and VaultHub	Not proved in this lane. If their values enter an accounting transition, they enter only through named, well-formed input assumptions; otherwise no theorem relies on them.
Gas, events, revert strings, and exact call-stack shape	Engineering and integration review surfaces, not conservation-law premises.

Allowed public wording

> **Precise claim**
>
> This report package demonstrates the executable local claim shape: the selected PR #1811 SRv3 accounting checks pass over a focused economic-state-machine model, under explicit assumptions for external contracts, accepted oracle inputs, cryptographic witnesses, governance configuration, and deployed-system refinement. Use broader theorem or deployed-system proof language publicly only after stronger artifacts are added and independently rebuilt.

4 Reproducibility and handoff

Reviewer command

A reviewer does not need to trust the PDF alone. The handoff includes the proof sources and commands that rebuild the selected executable target set and check that the PDF's assumptions match the machine-readable trust-boundary report.

The rebuild is the reviewer's receipt. If it succeeds, the reviewer learns that the target IDs in the PDF exist, the executable checks pass, and the listed assumptions match the machine-readable trust report. If it fails, the PDF claim is not ready to rely on.

```
git clone git@github.com:lfglabs-dev/lido-srv3-proof-closure.git
cd lido-srv3-proof-closure
make bootstrap
make test
make prove
make report
```

These commands check all selected model tests, verify every matrix row has a matching artifact in `proofs/`, emit `proofs/logs/proof-report.json`, and rebuild the PDF.

Minimum provenance fields

Field	Repository content
PR and source revision	PR #1811 URL, base branch, audited commit hash, and review timestamp. Report package target: `d088bbc2deac9913b68036d73d35c37aa6279b90`.
Proof toolchain	Standard-library Python executable harness, pinned Verity reference `33722270d996c7a3a520a71ecee42d7d232da100`, and `make test` / `make prove`.
Target namespace	Selected proof target files in `proofs/`, target names in `verity/targets/srv3-proof-targets.json`, and the command that fails on a failed model check.
Trust allowlist	Machine-readable assumption IDs, out-of-model boundaries, and target-to-assumption mapping in `verity/targets/trust-boundary.json`.
Artifact integrity	Generated PDF in `dist/lido-srv3-formal-methods-report.pdf` and proof output in `proofs/logs/proof-report.json`.

Source anchors

The matrix uses broad PR #1811 surfaces to stay readable. The exact source map is `verity/targets/source-map.json`. Anchors for the target source revision include:

- Buffer and reserve accounting: `Lido.sol::_getBufferedEtherAllocation`, `getDepositableEther`, and `withdrawDepositableEther`.
- Deposit allocation: `StakingRouter.sol::deposit` and `SRLib.sol::_getDepositAllocations`.
- Reports and rewards: `StakingRouter.sol::reportValidatorBalancesByStakingModule`, `getStakingRewardsDistribution`, and `SRLib.sol::_reportRewardsMinted`.
- Oracle validation: `AccountingOracle.sol::_checkStakingRouterModuleBalances`.

Handoff contents

- **PDF:** `dist/lido-srv3-formal-methods-report.pdf`. This report package for Lido reviewers.
- **Lockfile:** `proofs/LOCKFILE.md`. Pinned Lido and Verity references.
- **Trust report:** `verity/targets/trust-boundary.json`. Machine-readable assumption and boundary register.
- **Proof targets:** `proofs/`. One target file for each selected P0 property.
- **Model:** `verity/src/verity_srv3/`. Executable SRv3 economic state machine.
- **Reference tests:** `tests/solidity-reference/` and `tests/verity/`. Copied Lido references and executable mirrors.

Integrity checks

Check	Expected result
`make test`	All executable Verity-style mirror tests pass.
`make prove`	Bounded proof harness passes SRV3-P1 through SRV3-P6 and writes `proofs/logs/proof-report.json`.
`make report`	Rebuilds `dist/lido-srv3-formal-methods-report.pdf`.
`proofs/LOCKFILE.md`	Contains Lido PR #1811 commit `d088bbc2deac9913b68036d73d35c37aa6279b90` and Verity commit `33722270d996c7a3a520a71ecee42d7d232da100`.

A Appendix A: Reader glossary

Router The SRv3 component that allocates deposit work across staking modules and records aggregate module accounting.

Module A staking-provider lane managed by the router.

Accepted report Oracle balance data that has passed the modeled SRv3 validation checks and may update accounting state.

Withdrawal reserve ETH kept aside for withdrawals, not available for new deposits.

CL Consensus layer, the Ethereum system that records validator activity.

Lean A proof checker that can verify theorem statements. This package does not ship Lean theorem artifacts.

Verity The modeling reference pinned for this work. This repository ships a local Verity-style executable economic model.

P0 Highest-priority property class. Here it means an accounting claim important enough that the executable package should either check it or clearly say why it remains open.

Proof target A precise executable check tied to source anchors, assumptions, model files, and proof logs.

Assumption A named premise that the executable target may use but does not prove.

Trust boundary The edge of the proof: what the model proves, what it assumes, and what remains for separate review.

B Appendix B: Certora delta

This lane does not replace earlier Certora work. It shows how a PR #1811 proof-closure package continues from that work: moving from bounded or summarized checks toward explicit SRv3 accounting transitions, balance-based module reasoning, and named assumptions that a reviewer can audit.

Legacy coverage class	Disposition	Final treatment
V2 validator-count accounting	Continued	Reframed as module-balance conservation over accepted per-module balances.
V2 deposit availability bounds	Re-expressed	Expressed as reserve separation plus exact 32 ETH pull accounting.
V2/V3 reward distribution checks	Re-expressed	Expressed as accepted-balance reward computation, fee bounds, and recipient alignment.
Router status-gating assertions	Reused and extended	Applied to active, stopped, and deposit-paused gates in the modeled economic paths.
V3 bounded module-array specs	Mirrored	Executable checks cover named fixtures and bounded enumerations with named well-formedness assumptions.

C Appendix C: Expected axiom and assumption summary

Class	Expected final value	Interpretation
Lean axioms introduced by LFG	N/A	This package does not ship a Lean namespace.
Admitted theorem bodies	N/A	This package does not ship theorem bodies with `sorry` or `admit`.
Named model assumptions	5	The report package assumes this allowance: external summaries, beacon sink behavior, accepted oracle reports, module identity/order well-formedness, and arithmetic domain.
Selected falsification/regression harness counterexamples	0	`make prove` reports no selected P0 counterexample trace in the named bounded harness. This is not a claim about unmodeled deployed-system behavior.

D Appendix D: Follow-on lanes

These lanes are recommended because they touch adjacent SRv3 risk surfaces, not because they are required to support the accounting claims in this report package. Recommended sequence after this P0 accounting closure:

1. ExitBus authorization, replay resistance, duplicate prevention, and triggerable-exit fee/refund exactness.
2. Consolidation fee conservation, source and target binding, and source-ownership caveats.
3. Top-up and consensus-layer proof binding.
4. Oracle sanity limits, drain guards, and second-opinion oracle composition.
5. Deployment governance, role graph, and exact migration refinement.
6. VaultHub F-01/F-02/F-03, if still relevant to the active review path.